Optimizing Reverse Time Migration Performance for Seismic Imaging by High-speed High-fidelity Lossy Compression

Anonymous Author(s)

ABSTRACT

Seismic imaging is an exploration method estimating the seismic characteristics of the earth's sub-surface for geologists and geophysicists, which can also be used to explore mineral, coal, gas, oil for fuel companies. Reverse time migration (RTM) is a critical method in the seismic imaging analysis and it can produce extremely large volumes of data that need to be kept in storage for later use during its execution. The traditional solution transfers the vast amount of data to peripheral devices and loads them back to memory whenever needed, which may cause substantial burden to I/O and storage space. As such, an efficient data compressor turns out to be a very critical solution. In order to get the best overall RTM execution performance, we develop a novel hybrid lossy compression method (called HyZ), which is not only fairly fast in both compression and decompression but also has a good compression ratio with satisfactory reconstructed data quality for post hoc analysis. Specifically, this hybrid compression method combines a high-speed block-wise regression based compression method (called BR) and a fast bit-manipulation based compression method, which can obtain fairly good quality for both RTM snapshots and final stacking image. We evaluate many state-of-the-art error-controlled lossy compression algorithms (including HyZ, BR, SZx, SZ, ZFP, cuSZ, cuZFP, etc.) in heterogeneous environments. Experiments show that HyZ can significantly improve the overall performance for RTM by xx-xx× because of its good compression quality and high execution speed.

KEYWORDS

Lossy Compression; Performance; Seismic Imaging; Reverse Time Migration

1 INTRODUCTION

Seismic imaging is an exploration method used to estimate the seismic characteristics of the earth's sub-surface by measuring the reflected acoustic energy waves. This technology has been broadly used to explore the sub-surface structure of rock formations for geologists and geophysicists, or used to explore mineral, coal, gas, oil for fuel companies. Reverse time migration (RTM) is a cutting-edge seismic imaging method, which has been widely used in the seismic imaging community.

RTM is a both computation- and data-intensive task. Specifically, RTM involves two critical stages – a forward propagation of source wavefield and a backward propagation of receiver wavefield. Each of these two stages requires massive wave-processing computations. During the forward propagation, RTM would generate thousands of 3D snapshots. These 3D snapshots occupy extremely large volumes of data and should be accessible for later use in RTM execution. After being analyzed by backward propagation, these consecutive 3D snapshots produced by the forward propagation will be combined into a stacking image, which is a final analytic result to reveal the structural information of the sub-surface. With the development of parallel programming models (e.g. OpenMP and MPI) and GPU implementation supports (CUDA), the computation costs of RTM are significantly reduced. However, how to efficiently manage such a large volume of 3D snapshot data remains a critical challenge.

To address the above data management issue, a straight-forward solution is transferring the snapshots from memory to peripheral devices temporarily and loading them back to memory upon usage, which however still faces some serious issues or challenges. On the one hand, using peripheral device to store the vast amount of runtime data may degrade the overall execution performance because of the expensive data transferring cost inevitably. On the other hand, the capacity of the peripheral device may still be not enough for the extremely large amount of data to be produced during the forward propagation. For example, this peripheral device could be a parallel file system (PFS), which projects only tens of TBs for a regular user on a supercomputer [22].

Another potential solution is compressing the RTM snapshot data to mitigate data transfer cost and storage burden. Specifically, the snapshots produced by the forward propagation are compressed by a data compressor, and the compressed snapshots would be kept in either memory or peripheral devices. Later on, the compressed snapshots will be loaded and decompressed for the backward propagation analysis at runtime. As revealed by many existing studies [24], lossless compression suffers from very low compression ratios (1.2~2 in general) for scientific datasets, so we focus on error-controlled lossy compression in our work. In order to prevent the RTM execution from being delayed significantly by compression/decompression overhead, the qualified compressor must offer very high compression and decompression throughput. This is because the RTM method generally has quite high performance in its execution [14] due to substantially optimized parallel code. To the best of our knowledge, we are the first work to integrate lossy compression to RTM execution framework.

In this paper, we explore the best lossy compression method that has high speed in both compression and decompression, and also projects good compression quality (high ratio and high fidelity of data), such that the overall RTM execution can be improved significantly without any loss of analysis quality. The key contributions are summarized as follows.

- We develop a lossy compression based RTM method for seismic imaging analysis, in order to deal with the vast amount of data produced during the parallel execution more efficiently.

Conference'17, July 2017, Washington, DC, USA
2022. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

- We propose a novel lossy compression method – called *HyZ* which combines two high-speed lossy compression algorithms. The first one is our proposed block-wise regression-based compressor (BR), and the other one is SZx [30].
- Our hybrid lossy compressor can make the best use of their advantages and bypass their disadvantages to obtain very high compression and decompression speeds, meanwhile preserving high qualities for both snapshot data and final stacking image results.
- We integrated different lossy compressors (including HyZ, BR, SZx, SZ, ZFP, SZInterp, etc.) into a production-level parallel RTM code and run it with thousands of snapshots on a supercomputer.
- We comprehensively investigate the quality for each of the lossy compressors and their impact to RTM execution results as well as overall parallel performance.
- Experiments show that our proposed hybrid compressor has a top-tier performance in class, also leading to good compression ratios (5+) for users. Specifically, our solution that combines BR and SZx significantly outperforms the second-tier lossy compressors (SZ, ZFP, etc.) by up to xx% in compression throughput. Our proposed high-speed lossy compression method (BR) obtains a much better quality in reconstructed snapshot data than SZx does. Under our solution, the overall parallel execution performance can be improved by xx-xx× compared with the execution without lossy compressor, and by 120% over the execution deployed with the second-tier lossy compressors.

The rest of the paper is organized as follows. In Section 2, we discuss the related work. In Section 3, we formulate the research problem. In Section 4, we first analyze an existing top-tier high-performance lossy compression algorithm SZx and explain its limitation in RTM execution, then propose BR compressor along our hybrid solution HyZ. In Section 5, we comprehensively evaluate many state-of-the-art compressors by running a parallel RTM code used in industry. In Section 6, we conclude the paper with a discussion of the future work.

2 RELATED WORK

In this section, we introduce the related works from two perspectives: existing solutions to resolve the big snapshot data issue in RTM and existing lossy compression algorithms that have been developed and used widely.

2.1 Resolving the Limited Memory Capacity Issue in RTM

The RTM analysis for seismic imaging is facing a serious memory burden in practice due to terabytes of snapshot data being generated during forward propagation. To overcome this bottleneck, several solutions from different perspectives have been proposed. Fu et al[12] adopted an FPGA-based solution to remove memory constraints and provide a high performance. Perrone et al [23] designed a domain-specific data partition strategy to parallel RTM execution on main memory of different nodes and thus avoid the low I/O bandwidth of disk. AlOnazi et al [4] addressed this issue by deploying executions on distributed-memory systems equipped with multiple GPUs. Alturkestani et al. [5] leveraged the GPU's High Bandwidth Memory (HBM) as an additional storage media layer to maximize RTM I/O Bandwidth. Although these existing methods achieve promising performance in RTM, the high storage requirement is not mitigated at all, as the size of generated image data remains the same. Several works [12, 23] have explored the possibility of using data compression techniques to trade for the I/O bandwidth and storage cost, while they mainly focus on the lossless compression in their solutions. Many existing studies [25] showed that lossless compressors [8, 9, 35] suffer from very low compression ratios, especially in a comparison with lossy compressors on scientific datasets.

2.2 Error-controlled Lossy Compression for Scientific Datasets

Error-controlled lossy compression is a very promising solution to resolve the data management issue in RTM execution. The most important advantage of lossy compression is significantly higher compression ratio than lossless compression, as demonstrated in prior studies [11, 27]. The existing state-of-the-art lossy compressors include SZ [11, 27], ZFP [16], FPZIP [17], TTHRESH [6], MGARD [3], bit grooming [31], digit rounding [10], and several emerging auto-encoder based compressors [13, 18, 20]. In the scientific data lossy compression community, SZ and ZFP are two leading compressors, because of their fairly high compression ratios and high compression speed compared with other state-of-the-arts. According to [11], SZ and ZFP have much higher compression ratios than FPZIP because their innovative algorithms in data prediction and decorrelation. Based on the SZ compression framework, Zhao et al. [32] developed a very effective prediction method that can significantly improve the compression ratio on RTM analysis datasets. However, its speed is comparable or even lower than that of the generic SZ compressor, which does not meet the high-speed requirement in our use-case. Although TTHRESH can get much higher compression ratios than SZ and ZFP do, it suffers from very low compression/decompression performance (about one order of magnitude lower) due to its expensive high order singular value decomposition (HOSVD). Bitgrooming and digitrounding both suffer from very low compression ratios because they both ignore the data correlations in compression. There have been a few auto-encoder based lossy compression methods [13, 18, 20] proposed recently, but none of them are qualified for RTM executions because of very low performance in both compression and decompression. Specifically, Liu et al. [18] proposed an effective method to combine the auto-encoder and SZ compression framework which exhibits the best compression quality from among all auto-encoder based compressors, but it is still about 2-3× slower than SZ and ZFP.

3 SYSTEM DESIGN AND PROBLEM FORMULATION

In this section, we describe our designed RTM method integrated with error-controlled lossy compression technologies, and also formulate the research problem.

3.1 Design of Lossy-compression based RTM

Figure 1 illustrates the workflow of a parallel RTM execution code, which corresponds to the practical seismic imagining analysis. Specifically, at the beginning of each run, the RTM code reads a configuration file which contains several key parameters such as problem size, initial background data file, total number of snapshots, and how many time steps a snapshot will be saved for backward propagation analysis. The forward propagation of source wavefield generates a snapshot at each time step; while only a subset of the snapshots (e.g., every K time steps) are selected/kept for the later analysis (see step ❶ in Figure 1) and others are disregarded. In the traditional design, the selected snapshots are kept either in memory (if memory capacity is large enough) or dumped to external devices temporally. After the forward propagation, the execution enters the backward propagation of the receiver wavefield for the analysis of sub-surface structure, which depends on the snapshots generated by the forward propagation. As such, the snapshots saved during the forward propagation need to be reloaded for analysis (see step ❷ in Figure 1). A final stacking image (as the analysis result) is generated after the backward propagation (see step ❸ in Figure 1).

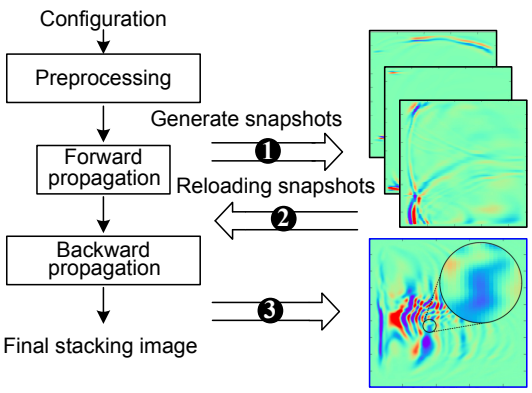


Figure 1: Illustration of Reverse Time Migration (RTM) Method

Since the parallel RTM executions produce extremely large amount of snapshots during the forward propagation, we manage to integrate lossy compression techniques in a scalable parallel RTM code. Specifically, unlike the traditional code which keeps the original raw snapshots either in memory or external devices such as parallel file systems (PFS), our design compresses the snapshots by a lossy compression method before saving them. During the backward propagation stage, whenever a snapshot needs to be used, the corresponding compressed data is loaded into the memory and decompressed for the analysis.

3.2 Problem Formulation

Our objective is to optimize the overall end-to-end execution performance of the RTM method, covering the cost of preprocessing, forward propagation, backward propagation as well as all possible overheads such as compression time and I/O cost. The fundamental idea is to leverage lossy compression to reduce the volume of forward propagation snapshot data, which thus can significantly reduce I/O cost for the overall execution.

Although lossy compression can obtain significantly higher compression ratios than lossless compression, it may introduce data distortion to the reconstructed data, thus we need to control the compression errors carefully especially from the perspective of the post hoc seismic analysis. As for the compression quality, we will focus on both compression ratio and quality of the reconstructed data. On the one hand, we use commonly-used lossy compression related metrics such as peak signal to noise ratio (PSNR) and structural similarity index measure (SSIM) to check the data distortion from the perspective of lossy compression. On the other hand, we also evaluate the visual quality of the reconstructed data as Seismic imaging analysts often need to analyze the waves by observing the snapshots and stacking images [1, 7].

4 HYZ: A HIGH-SPEED HIGH-FIDELITY LOSSY COMPRESSION METHOD FOR RTM

In this section, we develop a novel high-speed high-fidelity lossy compression method (called HyZ), which can also maintain a satisfactory compression ratio, for RTM execution and post hoc analysis.

Since RTM method generally has a relatively high parallel performance in seismic imaging analysis, a qualified lossy compressor has to be fast enough and also with good compression ratios. To this end, we first analyze an existing ultra-fast lossy compression algorithm (called SZx [30]) and explain its limitations in RTM execution. We observe that SZx cannot maintain high-fidelity in reconstructing single snapshot images within a satisfactory compression ratio (5+). To solve this issue, we then develop a high-speed regression-based lossy compressor (BR) that can preserve the quality of the snapshot data very well. Finally, to enable error-control in BR compressor, we propose our hybrid compressor design – HyZ, which integrates SZx to compress blocks that exceed the error bound in BR compressor during RTM execution.

4.1 Discussing the Limitations of SZx

One ultra-fast lossy compressor that is potentially suitable for RTM execution is SZx, which was proposed by Yu et al. [30]. The design guideline of SZx is making the compressor composed of fairly light-weight operations including only bitwise, addition, and subtraction. By comparison, other state-of-the-art lossy compressors depend on relatively heavier operations such as multiplication (used by SZ, ZFP), variable-length encoding (adopted by SZ, ZFP, MGARD, digit rounding, and bit grooming), and dictionary encoding (adopted by SZ, MGARD, digit rounding, and bit grooming).

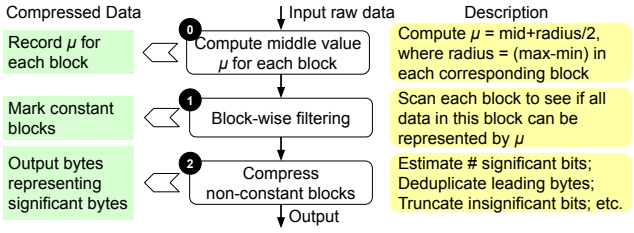


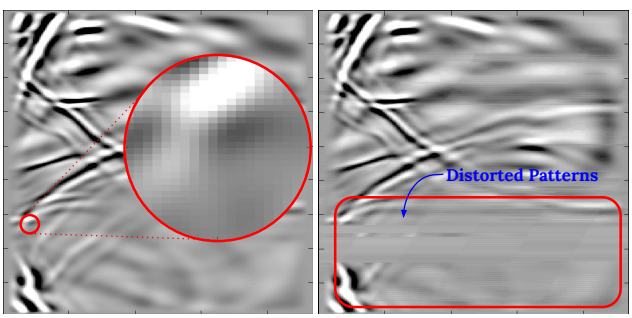
Figure 2: Compression pipeline (workflow) of SZx

The SZx algorithm splits the whole dataset into many small 1D fixed-size segments (or blocks), and then performs compression

on each block separately. It consists of two critical stages to process different data points in the dataset: ❶ block-wise filtering, ❷ compressing non-constant blocks, as illustrated in Figure 2.

Block-wise filtering aims to check each block to see if it can be represented by some constant number μ. Specifically, if the amplitude of the data variation in one block is lower than or equal to twice of the user-specified error bound, the data in this block can be approximated by μ=(max − min)/2, where min and max refer to the minimal value and maximum value in the corresponding data block, respectively. Such blocks are called 'constant blocks'; otherwise, they are 'non-constant blocks'. As such, the block-wise filtering step requires a preprocessing step to compute the middle value μ for each block (shown as step ⓪ in Figure 2). After finishing the block-wise filtering step, there will be two outputs committed to the compressed data: the μ array and the block-type array, which record the middle values and block types, respectively.

Such a block-wise filtering method can significantly improve the compression ratio, however, it may reduce the visualization quality for RTM snapshot data. The constant blocks can be reconstructed only based on the μ array and the RTM snapshot data is routinely very smooth in space so that majority of the data belong to the constant blocks. Though such design can improve compression ratio in RTM data, indiscriminately using the same μ to represent the whole block is likely to cause visible artifacts.



(a) Original Data (b) SZx, REL=1E-2 (PSNR: 54.95dB, SSIM: 0.72, CR: 5.12)

Figure 3: Visualization of single snapshot image (time step=3000) by SZx with REL 1E-2 error bound (~5 compression ratio) and 128 block size choice.

To verify such situation, Figure 3 visualizes reconstructed single snapshot (time step=3000) produced by SZx. We set error bound as REL 1E-2 (*c.f.* Section 5.1.4) to keep the compression ratio at around 5, which is the minimal satisfactory ratio in RTM execution. As shown in Figure 3 (b), a large area of distorted patterns can be observed, due to the compression algorithm design for 'constant blocks' in SZx. Also, the PSNR and SSIM are only 54.95dB and 0.72, respectively. Improving visualization quality requires smaller error bound such as REL 1E-3 or smaller block size choice such as 32, which will also cause lower compression ratio and cannot satisfy the seismic community. In conclusion, the ultra-fast design of SZx is suitable in speed for RTM execution, however, it has some defects with respect to data visualization.

4.2 Block-wise Regression based Lossy Compression (BR)

The key idea is splitting the entire dataset into many small blocks (e.g., 4×4×4) and then approximating the data values in each block by a regression hyperplane. This idea is motivated by an important observation that the data points in a very small region in space are likely able to be approximated by a simple hyperplane (e.g., linear regression or quadratic regression). We demonstrate three examples in Figure 4: block Ⓐ and block Ⓑ exhibit linear hyperplane property and block Ⓒ exhibits quadratic hyperplane property.

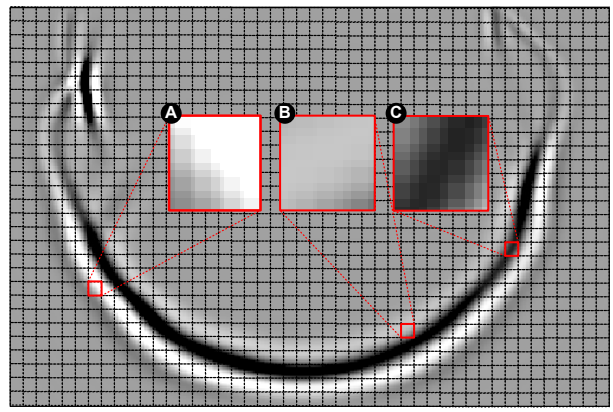


Figure 4: Illustration of Linearality of Small Data Blocks in An RTM Snapshot

In our design, we explore both linear regression and quadratic regression, respectively, as described below.

- **Linear Regression**. During the compression, each data block is approximated by a linear hyperplane with four coefficients ($f(x,y,z)$=$\beta_1 x + \beta_2 y + \beta_3 z + \beta_0$), and thus only four coefficient values need be stored to substitute all the data in that block. During the decompression, each data block would be recovered by the hyperplane reconstructed with the four regression coefficients. Without loss of generality, suppose the data block size is n_3×n_2×n_1, then its regression coefficients can be calculated as the following formula (d_{ijk} refers to the data values in the data block at the location {i,j,k}).

$$\begin{cases} \beta_1 = \frac{6}{n_1 n_2 n_3 (n_1+1)} (\frac{2V_x}{n_1-1} - V_0) \\ \beta_2 = \frac{6}{n_1 n_2 n_3 (n_2+1)} (\frac{2V_y}{n_2-1} - V_0) \\ \beta_3 = \frac{6}{n_1 n_2 n_3 (n_3+1)} (\frac{2V_z}{n_3-1} - V_0) \\ \beta_0 = \frac{V_0}{n_1 n_2 n_3} - (\frac{n_1-1}{2}\beta_1 + \frac{n_2-1}{2}\beta_2 + \frac{n_3-1}{2}\beta_3) \end{cases} \quad (1)$$

where $V_0 = \sum_{i=0}^{n_1-1}\sum_{j=0}^{n_2-1}\sum_{k=0}^{n_3-1} d_{ijk}$, $V_x = \sum_{i=0}^{n_1-1}\sum_{j=0}^{n_2-1}\sum_{k=0}^{n_3-1} i * d_{ijk}$,

$V_y = \sum_{i=0}^{n_1-1}\sum_{j=0}^{n_2-1}\sum_{k=0}^{n_3-1} j * d_{ijk}$, $V_z = \sum_{i=0}^{n_1-1}\sum_{j=0}^{n_2-1}\sum_{k=0}^{n_3-1} k * d_{ijk}$.

In this compression method, the compression ratio can be estimated based on the following formula. For instance, when the block size is 4×4×4, the compression ratio is

64/4=16; when the block size is 3×3×3, the compression ratio is 27/4=6.75.

$$CR = \frac{n_3 n_2 n_1}{4} \tag{2}$$

- **Quadratic Regression**. Unlike the linear regression, the quadratic regression (a.k.a., 2nd order regression) constructs a quadratic hyperplane with 10 regression coefficients to fit the data values in each block (as shown in Formula (3)), such that the data values can be approximated more accurately especially when the block size is relatively large.

$$\begin{aligned} f(x,y,z) = \beta_0 + \beta_1 x + \beta_2 y + \beta_3 z + \beta_4 x^2 + \beta_5 xy \\ + \beta_6 xz + \beta_7 y^2 + \beta_8 yz + \beta_9 z^2 \end{aligned} \tag{3}$$

where x, y, and z denots the relative location (or index) of in the data block. That is, x=0,1,$\cdots$,n_1, y=0,1,$\cdots$,n_2, z=0,1,$\cdots$,n_3, for the data block $n_3 \times n_2 \times n_1$. Similar to the above compression method, the quadratic regression based compression needs to store 10 coefficients in each block, which will be used to reconstruct data during the decompression. Using the least-square method, the 10 regression coefficients can be calculated as the Equation (4).

$$\boldsymbol{\beta} = \left(\sum_{i=0}^{n_1-1} \sum_{j=0}^{n_2-1} \sum_{k=0}^{n_3-1} A \right)^{-1} \boldsymbol{V}^T \tag{4}$$

where

$$A = \begin{bmatrix} 1 & x & y & z & x^2 & xy & xz & y^2 & yz & z^2 \\ x & x^2 & xy & xz & x^3 & x^2y & x^2z & xy^2 & xyz & xz^2 \\ y & xy & y^2 & yz & x^2y & xy^2 & xyz & y^3 & y^2z & yz^2 \\ z & xz & yz & z^2 & x^2z & xyz & xz^2 & y^2z & yz^2 & z^3 \\ x^2 & x^3 & x^2y & x^2z & x^4 & x^3y & x^3z & x^2y^2 & x^2yz & x^2z^2 \\ xy & x^2y & xy^2 & xyz & x^3y & x^2y^2 & x^2yz & xy^3 & xy^2z & xyz^2 \\ xz & x^2z & xyz & xz^2 & x^3z & x^2yz & x^2z^2 & xy^2z & xyz^2 & xz^3 \\ y^2 & xy^2 & y^3 & y^2z & x^2y^2 & xy^3 & xy^2z & y^4 & y^3z & y^2z^2 \\ yz & xyz & y^2z & yz^2 & x^2yz & xy^2z & xyz^2 & y^3z & y^2z^2 & yz^3 \\ z^2 & xz^2 & yz^2 & z^3 & x^2z^2 & xyz^2 & xz^3 & y^2z^2 & yz^3 & z^4 \end{bmatrix}$$

$$\boldsymbol{V} = (V_1, V_2, .., V_9), V_t = \sum_{x=0}^{n_1-1} \sum_{y=0}^{n_2-1} \sum_{z=0}^{n_3-1} g_{xyz}(t) * f_{xyz}, 0 \le t \le 9$$

$g_{x,y,z}(t)$ returns tth element from list $[1, x, y, z, x^2, xy, xz, y^2, yz, z^2]$

In this method, the compression ratio can be calculated as follows, as there are 10 coefficients to store per block.

$$CR = \frac{n_3 n_2 n_1}{10} \tag{5}$$

Discussion. Although the regression method has been used in existing lossy compression techniques [15, 33], our solution substantially differ from the prior works. SZ2.1 [26] leverages the linear regression to predict the data values in the high-compression cases [15]. Zhao et al. [33] integrates the 2nd-order regression into the prediction stage of the SZ compression framework, which can significantly improve the compression quality for some wave-patterned datasets. These related works both treat the regression method as a predictor in the entire compression pipeline. In order to keep a very high compression ratio, they have to compress the coefficients, which may degrade the accuracy of the regression/prediction in turn. As such, they have to operate a few other expensive compression steps (such as quantization, variable-length encoding and dictionary encoding) to control the compression errors. By comparison, our solution can skip the three expensive compression steps in that we store the regression coefficients as they are in the compressed data. Such a lightweight algorithm has substantially higher compression/decompression speeds, while still preserving the reconstruction quality very well (to be shown later).

4.3 HyZ: Combining BR and SZx for RTM Execution

In this subsection, we describe how we construct a hybrid lossy compression framework HyZ by using two compression methods (BR and SZx).

BR splits RTM snapshot data into many small blocks and adopts a regression hyperplane to preserve data value inside each block, which preserve the reconstructed RTM data quality. By only storing the regression coefficients as compressed data, BR is also utlra-fast and suitable for RTM execution. However, due to the lightweight design, BR compressor does not support error-control. For example, we use BR-Linear with 4×4×4 block size to compress a single RTM snapshot data (time step=3000). After the reconstruction, we find there are around less than 1% data points (distributed in 3% blocks) exceed REL 1E-2 error bound. Existing error-control designs [26, 33] require time-consuming computations (such as quantization and encoding) that negatively impact the speed constraint in RTM execution. Thus, to further improve the reconstructed data quality for RTM, we integrate SZx into BR compressor and propose HyZ, of which workflow is shown in Figure 5.

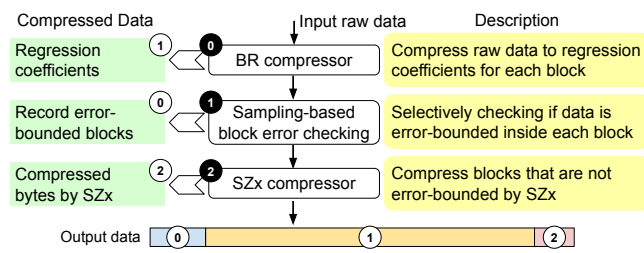


Figure 5: Compression pipeline (workflow) of HyZ

The fundamental design of HyZ is to enable a fast error-control in BR compressors via an another ultra-fast compressor SZx. Specifically, HyZ consists of 3 major steps: ⓪ BR compressor, ❶ sampling-based block error checking, and ❷ SZx compressor. Given a user-specified error bound and the input raw data, HyZ first utilizes BR compressor (shown as ⓪ in Figure 5) to process data and obtains the corresponding regression coefficients. In order to check if data points are error-bounded inside each block, HyZ then reverses the regression coefficients to the decompressed data. Since data comparison operations are likely to incur extra computation cost, HyZ samples data from eight corners inside a block (i.e. cube) to determine if this block is error-bounded (shown as ❶ in Figure 5). By doing so, xx% of outlier blocks can be found within only xx% extra overhead in our RTM execution. Though there are still xx% blocks are not error-bounded, we argue that these outlier data points only occupy xx% of the whole dataset, which in fact do not affect both statistic image quality (PSNR and SSIM) and visualization quality. Finally, for all the recognized blocks that are not error-bounded, HyZ combines them to an array and adopts SZx to compress this array (shown as ❷ in Figure 5). Note that although constant block

compression methodology in SZx may cause visible errors on RTM dataset, such circumstance only can be observed on consecutive data. However, in this combined array produced by ❶, the data points are actually not consistent with respect to their position in current snapshot. HyZ combines two ultra-fast compressors, hence not only can achieve promising visualization results but also maintain top-tier compression/decompression speeds.

5 PERFORMANCE EVALUATION

In this section, we present experimental setups and analyze the performance evaluation results.

5.1 Experimental Settings

5.1.1 Environment. We perform multi-thread OpenMP experiments using Argonne Bebop supercomputer, in which each node has two Intel Xeon E5-2695 v4 processors and 128 GB of DRAM. Since each Bebop node is equipped with 36 cores, we run our experiments with 36 threads for each compressor.

5.1.2 Simulation and Datasets. We perform the evaluation using one RTM simulation code developed by a seismic imagining team from industry [2]. We run the code with 3600 time steps at a problem size 449×449×235. That is, there are 3600 snapshots generated to compress during the forward propagation stage and each contains 449×449×235 single-precision floating-point values.

5.1.3 State-of-the-Art Compressors in Our Evaluation. We evaluate BR, HyZ, and SZx compressors in RTM simulation with four other state-of-the-art lossy compressors (SZ-Interp [32], SZ-Interp-fast, SZ2.1.12[26], ZFP0.5.5 [16]). SZ-Interp adopts an interpolation method in the SZ compression framework, which can significantly improve the compression quality in turn. SZ-Interp-fast is modified version from SZ-Interp, by replacing its expensive steps (Huffman encoding and Zstd) by a lightweight encoder - run-length encoding [29]. SZ2.1 is the latest public version of the SZ compressor, which exhibits fairly high compression quality and performance as validated by many prior studies [15, 27]. ZFP is another outstanding error-bounded lossy compressor, which may outperform SZ2.1 in some cases as verified by existing studies [32]. We have integrated all the lossy compressors into the RTM simulation in our experiments.

We also evaluate the compression qualities of three state-of-the-art lossless compressors – ZFP, FPZIP, and Zstandard (Zstd) for a comparison. ZFP and FPZIP also supports lossless compression in addition to lossy compression as long as all the bit-planes are preserved during the compression. Zstd is an outstanding lossless compressor, which is substantially faster than other lossless compressors according to prior studies [19, 35].

5.1.4 Error Bounds for Lossy Compressors. In lossy compressors with error-control, there are two types of error bounds, absolute (ABS) error bound and relative (REL) error bound. The ABS error bound (denote as δ) is set as a constant. As for REL error bound, it is a linear computation based on the global data value range size, i.e. λr, where $\lambda \in (0, 1)$ and r denote relative ratio and and data range size respectively. As a result, for a given set of data $D = \{d_1, d_2, ..., d_n\}$, its decompressed data $D' = \{d'_1, d'_2, ..., d'_n\}$ should satisfy error bounds by following equations.

$$\max_{i=1,2,...,n} (d_i - d'_i) \leq \begin{cases} \delta & \text{(ABS error bound)} \\ \lambda r & \text{(REL error bound)} \end{cases} \quad (6)$$

We use δ and λ to quantify ABS and REL error bounds for the following contexts.

5.1.5 Evaluation Metrics.

- Compression Ratio (CR): CR (defined as $\frac{original\ size}{compressed\ size}$) indicates how much the snapshot data can be reduced in memory. The minimum qualified compression ratio is 5, in that the execution node has 128GB memory and the total volume of snapshot data is up to 635GB, otherwise the simulation would crash because of out-of-memory issue.
- Compression speed and decompression speed: As mentioned, compression/decompression speed is critical to the overall execution performance, as too high overhead may cause significant delays unexpectedly. In general, they are measured in the form of throughput (i.e., $\frac{original\ size}{compression\ time}$ (GB/s) and $\frac{reconstructed\ size}{decompression\ time}$ (GB/s), respectively).
- PSNR and SSIM: as mentioned in Section 3, they are used for quantifying data distortion in RTM simulation. Their detailed definitions can be found in [28] and [34].
- Visual quality: We will visualize the data for both snapshots and stacking image to present the impacts of different lossy compressors to the simulation results.
- Overall performance of RTM: Entire execution time of one RTM simulation run that involves all the three stages: data preprocessing, forward propagation, backward propagation, and stacking image file generation.

5.2 Evaluation Results and Analysis

We present and analyze evaluation results as follows.

5.2.1 Compression Ratio. Table 1 presents the average compression ratios of different lossy compressors in 3600 snapshots with the same error bounds. For error-bounded compressors (such as SZx, SZ and ZFP), we use two types of error bounds – absolute error bound (ABS) and value-range based relative error bound (REL), respectively, as they are both commonly used in scientific applications [15, 21, 27]. Their specific definitions can be found in [28]. For the non-error-bounded compressor BR, we present its compression ratios based on different block sizes. We can clearly observe that all lossy compressors here meet the compression ratio requirement (5+) even when we set relatively low error bounds: e.g., REL=4E-4 and ABS=4E-6. Specifically, SZ-Interp exhibits the highest compression ratios (113.2~213.53), and SZx has lowest compression ratios (6.22~9.57). The key reason is that SZ-Interp fully leveraged the data correlation in 3D dimensions by a dynamic interpolation method [32], while SZx is designed based on 1D correlation of the dataset. BR-Linear obtains over 2× higher compression ratios than BR-Quadratic does, as it keeps fewer coefficients per block than BR-Quadratic during the compression phase. SZ-Interp-fast has significantly lower compression ratios than SZ-Interp , in that its run-length encoding is not as effective as Huffman encoding especially when the error bound is relatively low (the pattern with

consecutive symbols is very rare in this situation). We notice that SZ has different compression ratios between OpenMP and Serial (i.e. single thread) versions, and the compression ratio with OpenMP version is degraded by ~2.5× on average. This is because the OpenMP version needs to make sure data independence across blocks during compression, which would lose prediction accuracy inevitably.

Table 1: Average compression ratio based on the same error bound over 3600 snapshots in RTM simulation. (All compressors are OpenMP versions except for SZ (serial))

Compressor	Relative Error Bound 1E-3	4E-4	Absolute Error Bound 1E-5	4E-6
SZx	7.58	6.22	9.57	7.72
SZ-Interp	213.53	116.60	177.92	113.20
SZ-Interp-fast	17.05	9.09	16.26	11.06
SZ	26.94	25.51	26.83	24.37
SZ(Serial)	70.36	50.68	81.47	54.94
ZFP	26.92	20.65	38.74	32.04
Compressor	Block Size $3\times3\times3$	$4\times4\times4$	$5\times5\times5$	$6\times6\times6$
BR-Linear	6.66	15.72	31.11	52.64
BR-Quadratic	2.67	6.29	12.44	21.06
Compressor	Threshold e 1E-1	1E-2	1E-3	1E-4
HyZ				

For the lossless compressors, as shown in Table 2, the overall compression ratios are in the range of [1.53, 2.17], which is too low to complete the whole simulation because of limited memory capacity. Moreover, they are all considerably slower than lossy compressors: the serial versions of FPZIP, ZFP and Zstd are slower than SZ(serial) by ~3-5×, ~5-10×, 10+×, respectively, which is far less than the expected compression throughput.

Table 2: Average compression ratio of three lossless compressors over 3600 snapshots in RTM simulation.

Compressor	Zstd	FPZIP	ZFP
Average CR	1.53	2.17	1.85

5.2.2 Compression and Decompression Speed. Table 3 and Table 4 present the OpenMP compression speed and decompression speed for all lossy compressors over 3600 snapshots in RTM simulation, respectively. As shown in the tables, the SZx and our proposed BR significantly outperform other lossy compressors. In absolute terms, the compression speeds of SZx and BR-Linear reach 12.09GB/s and 23.67GB/s on one node from Bebop supercomputer, which is about 2~4× as high as the state-of-the-art lossy compressors SZ and ZFP. For ZFP's decompression, it does not have OpenMP version, so we can only run the serial version instead, which suffers very low speed as shown in the table.

5.2.3 Data Distortion. We evaluate the data distortion for both final stacking image and individual snapshot images generated by RTM. As mentioned previously, we carefully evaluate two commonly-used lossy compression assessment metrics (PSNR and SSIM), and also analyze the visual quality from the perspective of users.

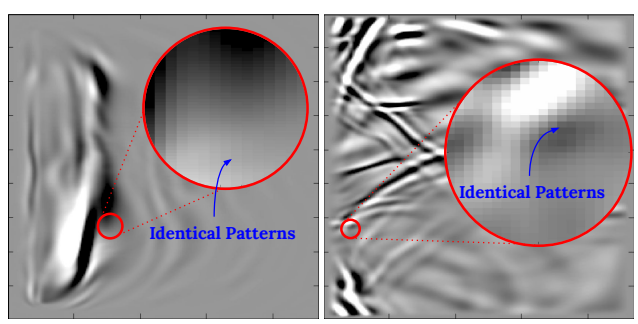
Figure 6 shows the visualization of the final stacking image and single snapshots produced by the RTM simulation with the HyZ compressor, where the user-specified threshold e is set to 1E-2. In

Table 3: OpenMP compression speeds (GB/s) based on the same error bound over 3600 snapshots in RTM simulation.

Compressor	Relative Error Bound 1E-3	4E-4	Absolute Error Bound 1E-5	4E-6	**Average**
SZx	12.48	11.08	16.00	8.78	12.09
SZ-Interp	2.87	2.76	2.76	2.47	2.72
SZ-Interp-fast	5.48	5.36	5.53	5.11	5.37
SZ	3.63	3.57	3.61	3.57	3.60
ZFP	2.82	2.56	4.33	4.04	3.44
Compressor	Block Size $3\times3\times3$	$4\times4\times4$	$5\times5\times5$	$6\times6\times6$	**Average**
BR-Linear	5.85	15.64	24.36	32.46	19.58
BR-Quadratic	1.73	3.48	5.36	5.94	4.13
Compressor	Threshold e 1E-1	1E-2	1E-3	1E-4	**Average**
HyZ					

Table 4: OpenMP decompression speeds (GB/s) based on the same error bound over 3600 snapshots in RTM simulation.

Compressor	Relative Error Bound 1E-3	4E-4	Absolute Error Bound 1E-5	4E-6	**Average**
SZx	19.65	18.13	22.45	15.44	18.92
SZ-Interp	5.26	4.88	4.77	4.48	4.85
SZ-Interp-fast	9.00	9.10	9.61	8.03	8.94
SZ	5.14	4.75	5.01	4.76	4.92
ZFP	0.60	0.54	0.62	0.58	0.59
Compressor	Block Size $3\times3\times3$	$4\times4\times4$	$5\times5\times5$	$6\times6\times6$	**Average**
BR-Linear	10.67	19.36	24.96	26.59	20.40
BR-Quadratic	1.95	2.63	5.62	6.08	4.07
Compressor	Threshold e 1E-1	1E-2	1E-3	1E-4	**Average**
HyZ					



(a) HyZ, Final Stacking Image (PSNR:115.62, SSIM:0.89, CR:8.81)

(b) HyZ, Single Snapshot, Time Step = 3000 (PSNR:60.87, SSIM:0.99, CR:6.67)

Figure 6: Visualization of final stacking image and single snapshot by HyZ with threshold of 1E-2

Figure 6 (a), it is observed that HyZ can maintain nearly identical data patterns compared with original data (see Figure **??** (a)) with a nice compression ratio 8.81. The corresponding PSNR and SSIM can reach up to 115.62dB and 0.89, respectively. Reconstructed single snapshots also exhibit a fairly high visual quality, as shown in Figure 6 (b).

5.2.4 Overall Performance of RTM. Figure 7 presents the overall performance of RTM simulation by integrating different lossy compressors. To clearly present the execution time, we breakdown each

RTM execution into four key stages: (1) Simulation Kernel Time: the core RTM algorithm, (2) I/O Write Time: writing/saving snapshot data during forward propagation including the compression time, (3) I/O Read Time: loading snapshot data in the backward propagation phase including the decompression time, (4) Stacking Image Generating Time: generating the final stacking image after backward propagation. According to the figure, running RTM with HyZ can get the best speedup in class (7.30-7.82×) compared with original simulation without compression techniques, because of high performance of both SZx and BR-Linear. In particular, SZx and BR-Linear obtain the speed-up of 7.49× and 6.93× on average, respectively. All the three compressors lead to top-tier performance. From among them, the HyZ is the best choice because it can preserve high data reconstruction quality for both snapshots and stacking image.

In contrast, the simulation runs with other compressors including BR-Linear, SZ-Interp-fast, SZ-Interp, SZ and ZFP would take notably longer time to finish. According to the figure, we can clearly observe that their I/O write times are all considerably higher than that of any top-tier performance compressor (HyZ, SZx, BR-Linear). This is due to the fact that they all suffer from substantially higher compression time (see Table 3). It is also observed that the RTM with ZFP has much higher wall-clock time than all the RTM executions with other compressors. The key reason is that ZFP supports only single-thread in the decompression, which may cause significantly longer total I/O read time in turn.

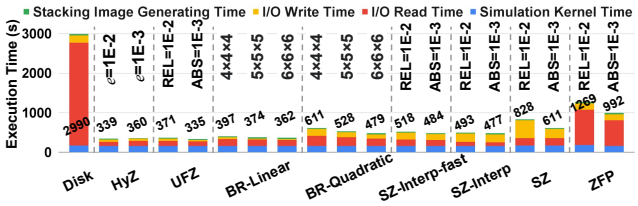


Figure 7: Overall performance of RTM with various integrated compressors. The 'Disk' bar means the baseline RTM simulation that offloads uncompressed snapshots to disk.

6 CONCLUSION AND FUTURE WORK

In this paper, we develop a hybrid lossy compression method (HyZ), in order to optimize the overall execution performance while preserving the data quality very well for users. We perform comprehensive experiments based on a total of 10 state-of-the-art lossless and lossy compressors. The key findings are summarized as below:

- Our solution HyZ combining BR and SZx has the top-tier performance in both compression and decompression, with satisfactory compression ratio and high fidelity reconstructed data in both snapshots and stacking image.
- BR exhibits higher quality than SZx on individual snapshots.
- SZx has a higher capability to preserve the final stacking image than BR.
- HyZ improves the overall performance of RTM from xx× to xx× over the original execution runs without using any lossy compression techniques.

In the future, we plan to further improve the RTM performance by leveraging lossy compression techniques to reduce the data communication cost across different ranks/devices at runtime.

REFERENCES

[1] [n. d.]. Reverse Time Migration (RTM) Technology. http://www.seismiccity.com/RTM.html.

[2] 2021. RTM code: The code information is hidden on purpose because of double-blind review policy; to be released after paper acceptance.

[3] Mark Ainsworth, Ozan Tugluk, Ben Whitney, and Scott Klasky. 2018. Multilevel techniques for compression and reduction of scientific data—the univariate case. *Computing and Visualization in Science* 19, 5-6 (2018), 65–76.

[4] Amani AlOnazi, Hatem Ltaief, David E. Keyes, I. Said, and Samuel Thibault. 2019. Asynchronous Task-Based Execution of the Reverse Time Migration for the Oil and Gas Industry. In *2019 IEEE International Conference on Cluster Computing, CLUSTER 2019, Albuquerque, NM, USA, September 23-26, 2019*. IEEE, 1–11.

[5] Tariq Alturkestani, Hatem Ltaief, and David E. Keyes. 2020. Maximizing I/O Bandwidth for Reverse Time Migration on Heterogeneous Large-Scale Systems. In *Euro-Par 2020: Parallel Processing - 26th International Conference on Parallel and Distributed Computing, Warsaw, Poland, August 24-28, 2020, Proceedings*. Springer, 263–278.

[6] Rafael Ballester-Ripoll, Peter Lindstrom, and Renato Pajarola. 2019. TTHRESH: Tensor Compression for Multidimensional Visual Data. *IEEE Transaction on Visualization and Computer Graphics* to appear (2019). arXiv:1806.05952.

[7] Allon Bartana, Dan Kosloff, Brandon Warnell, Chris Connor, Jeff Codd, David Kessler, Paulius Micikevicius, Ty Mckercher, Peng Wang, and Paul Holzhauer. 2015. GPU implementation of minimal dispersion recursive operators for reverse time migration. *SEG Technical Program Expanded Abstracts* 34 (2015), 4116–4120. https://doi.org/10.1190/segam2015-5754164.1 Publisher Copyright: © 2015 SEG.; null ; Conference date: 18-10-2011 Through 23-10-2011.

[8] Blosc compressor. 2018. http://blosc.org/. Online.

[9] M. Burtscher and P. Ratanaworabhan. 2009. FPC: A High-Speed Compressor for Double-Precision Floating-Point Data. *IEEE Trans. Comput.* 58, 1 (Jan 2009), 18–31.

[10] Xavier Delaunay, Aurélie Courtois, and Flavien Gouillon. 2019. Evaluation of lossless and lossy algorithms for the compression of scientific datasets in netCDF-4 or HDF5 files. *Geoscientific Model Development* 12 (09 2019), 4099–4113. https://doi.org/10.5194/gmd-12-4099-2019

[11] Sheng Di and Franck Cappello. 2016. Fast error-bounded lossy HPC data compression with SZ. In *2016 IEEE International Parallel and Distributed Processing Symposium*. IEEE, 730–739.

[12] Haohuan Fu and Robert G. Clapp. 2011. Eliminating the memory bottleneck: an FPGA-based solution for 3d reverse time migration. In *Proceedings of the ACM/SIGDA 19th International Symposium on Field Programmable Gate Arrays, FPGA 2011, Monterey, California, USA, February 27, March 1, 2011*. ACM, 65–74.

[13] Andrew Glaws, Ryan King, and Michael Sprague. 2020. Deep learning for in situ data compression of large turbulent flow simulations. *Phys. Rev. Fluids* 5 (Nov 2020), 114602. Issue 11. https://doi.org/10.1103/PhysRevFluids.5.114602

[14] Reynaldo Gomez. 2021. Reducing Costs with One-Pass Reverse Time Migration. https://developer.nvidia.com/blog/reducing-costs-with-one-pass-reverse-time-migration/.

[15] Xin Liang, Sheng Di, Dingwen Tao, Sihuan Li, Shaomeng Li, Hanqi Guo, Zizhong Chen, and Franck Cappello. 2018. Error-Controlled Lossy Compression Optimized for High Compression Ratios of Scientific Datasets. (2018).

[16] Peter Lindstrom. 2014. Fixed-rate compressed floating-point arrays. *IEEE Transactions on Visualization and Computer Graphics* 20, 12 (2014), 2674–2683.

[17] Peter Lindstrom and Martin Isenburg. 2006. Fast and efficient compression of floating-point data. *IEEE Transactions on Visualization and Computer Graphics* 12, 5 (2006), 1245–1250.

[18] Jinyang Liu, Sheng Di, Kai Zhao, Sian Jin, Dingwen Tao, Xin Liang, Zizhong Chen, and Franck Cappello. 2021. Exploring Autoencoder-based Error-bounded Compression for Scientific Data. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. 294–306. https://doi.org/10.1109/Cluster48925.2021.00034

[19] Jinyang Liu, Sihuan Li, Sheng Di, Xin Liang, Kai Zhao, Dingwen Tao, Zizhong Chen, and Franck Cappello. 2021. Improving Lossy Compression for SZ by Exploring the Best-Fit Lossless Compression Techniques. In *2021 IEEE International Conference on Big Data (Big Data)*. 2986–2991. https://doi.org/10.1109/BigData52589.2021.9671954

[20] Tong Liu, Jinzhen Wang, Qing Liu, Shakeel Alibhai, Tao Lu, and Xubin He. 2021. High-Ratio Lossy Compression: Exploring the Autoencoder to Compress Scientific Data. *IEEE Transactions on Big Data* (2021), 1–1. https://doi.org/10.1109/TBDATA.2021.3066151

[21] Tao Lu, Qing Liu, Xubin He, Huizhang Luo, Eric Suchyta, Jong Choi, Norbert Podhorszki, Scott Klasky, Mathew Wolf, Tong Liu, et al. 2018. Understanding and modeling lossy compression schemes on HPC scientific data. In *2018 IEEE*

International Parallel and Distributed Processing Symposium. IEEE, 348–357.

[22] ORNL. 2022. Summit supercomputer. https://www.olcf.ornl.gov/summit/.

[23] Michael Perrone, Lurng-Kuo Liu, Ligang Lu, Karen A. Magerlein, Changhoan Kim, Irina Fedulova, and Artyom Semenikhin. 2012. Reducing Data Movement Costs: Scalable Seismic Imaging on Blue Gene. In *26th IEEE International Parallel and Distributed Processing Symposium, IPDPS 2012, Shanghai, China, May 21-25, 2012*. IEEE Computer Society, 320–329.

[24] P. Ratanaworabhan, Jian Ke, and M. Burtscher. 2006. Fast lossless compression of scientific floating-point data. In *Data Compression Conference (DCC'06)*. 133–142. https://doi.org/10.1109/DCC.2006.35

[25] Seung Woo Son, Zhengzhang Chen, William Hendrix, Ankit Agrawal, Wei-keng Liao, and Alok Choudhary. 2014. Data compression for the exascale computing era-survey. *Supercomputing Frontiers and Innovations* 1, 2 (2014), 76–88.

[26] SZ2.1. 2022. https://github.com/szcompressor/SZ.

[27] Dingwen Tao, Sheng Di, Zizhong Chen, and Franck Cappello. 2017. Significantly improving lossy compression for scientific data sets based on multidimensional prediction and error-controlled quantization. In *2017 IEEE International Parallel and Distributed Processing Symposium*. IEEE, 1129–1139.

[28] Dingwen Tao, Sheng Di, Hanqi Guo, Zizhong Chen, and Franck Cappello. 2019. Z-checker: A framework for assessing lossy compression of scientific data. *The International Journal of High Performance Computing Applications* 33, 2 (2019), 285–303. https://doi.org/10.1177/1094342017737147

[29] Jiannan Tian, Sheng Di, Xiaodong Yu, Cody Rivera, Kai Zhao, Sian Jin, Yunhe Feng, Xin Liang, Dingwen Tao, and Franck Cappello. 2021. Optimizing Error-Bounded Lossy Compression for Scientific Data on GPUs. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. 283–293. https://doi.org/10.1109/Cluster48925.2021.00047

[30] Xiaodong Yu, Sheng Di, Kai Zhao, jiannan Tian, Dingwen Tao, Xin Liang, and Franck Cappello. 2022. SZx: an Ultra-fast Error-bounded Lossy Compressor for Scientific Datasets. arXiv:cs.DC/2201.13020

[31] Charles S. Zender. 2016. Bit Grooming: statistically accurate precision-preserving quantization with compression, evaluated in the netCDF Operators (NCO, v4.4.8+). *Geoscientific Model Development* 9, 9 (Sept. 2016), 3199–3211.

[32] Kai Zhao, Sheng Di, Maxim Dmitriev, Thierry-Laurent D. Tonellot, Zizhong Chen, and Franck Cappello. 2021. Optimizing Error-Bounded Lossy Compression for Scientific Data by Dynamic Spline Interpolation. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. 1643–1654. https://doi.org/10.1109/ICDE51399.2021.00145

[33] Kai Zhao, Sheng Di, Xin Liang, Sihuan Li, Dingwen Tao, Zizhong Chen, and Franck Cappello. 2020. Significantly Improving Lossy Compression for HPC Datasets with Second-Order Prediction and Parameter Optimization. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '20)*. Association for Computing Machinery, New York, NY, USA, 89–100. https://doi.org/10.1145/3369583.3392688

[34] Zhou Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13, 4 (April 2004), 600–612. https://doi.org/10.1109/TIP.2003.819861

[35] Zstandard. 2018. http://facebook.github.io/zstd/. Online.